

CONTROLADOR PERSONALIZADO DE AUTOESCALADO HORIZONTAL EN AWS

Diseño, implementación y validación experimental de un controlador propio, incluyendo el diagnóstico y corrección de una falla real detectada durante las pruebas

Estudiante: Emanuel Gonzalez Quintero

Curso: SI3016 - Cloud Computing

Institución: EAFIT

Fecha: 22/09/2026

Repositorio: <https://github.com/EmanuelGoQ/controller-challenge>

Región AWS: us-east-1 (N. Virginia)

1. Introducción

El objetivo de este trabajo fue diseñar e implementar un controlador propio de auto-escalado horizontal para una aplicación web desplegada sobre Amazon EC2, sin delegar la decisión a una política administrada de Auto Scaling Groups. Se construyó un programa en Python (controller.py) que observa métricas reales de la infraestructura a través de Amazon CloudWatch, analiza la demanda mediante un esquema de histéresis con ventanas de evaluación consecutivas, decide entre tres acciones explícitas (MAINTAIN_CAPACITY, INCREASE_CAPACITY, REDUCE_CAPACITY) y ejecuta directamente los cambios necesarios sobre EC2 y el Target Group de un Application Load Balancer (ALB).

A diferencia de un enfoque puramente teórico, la validación se realizó completamente sobre infraestructura real de AWS Academy Learner Lab, generando carga sintética real contra un endpoint CPU-bound. Durante esa validación se presentó una falla genuina de ejecución que, en cadena, provocó una caída temporal del servicio; esta situación se diagnosticó, corrigió y se documenta íntegramente en este informe como evidencia adicional de análisis de resiliencia.

2. Objetivos

2.1 Objetivo general

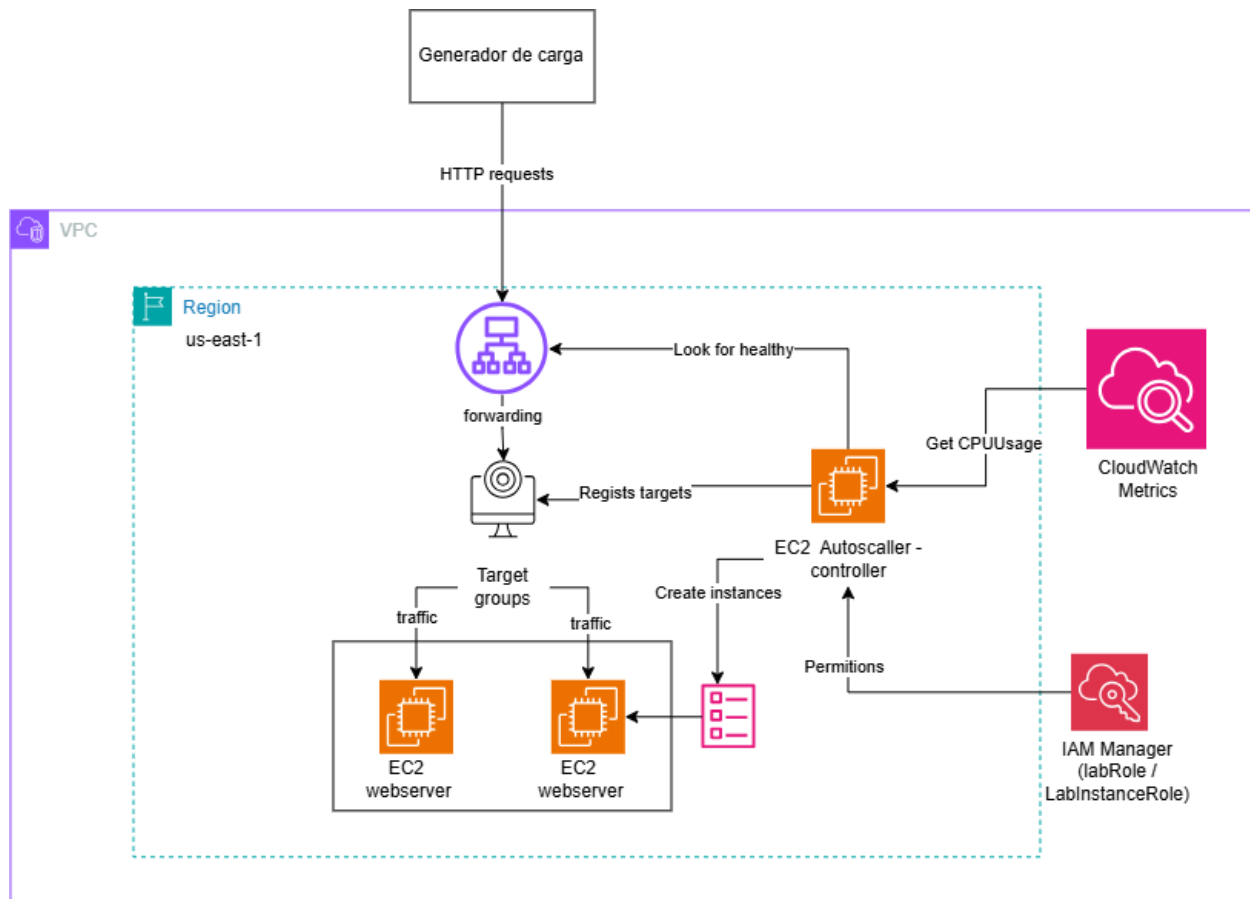
Construir y validar un controlador personalizado, ejecutado como servicio del sistema operativo, capaz de modificar horizontalmente la cantidad de instancias EC2 de una aplicación web según el uso de CPU observado en CloudWatch, respetando límites de capacidad y evitando decisiones inestables.

2.2 Objetivos específicos

- Desplegar una aplicación web (nginx) sobre instancias EC2 y distribuir el tráfico mediante un Application Load Balancer.
- Consultar la métrica CPUUtilization de EC2 desde Python utilizando boto3, y registrar TargetResponseTime del ALB como métrica complementaria de contexto.
- Implementar tres decisiones explícitas mediante histéresis: MAINTAIN_CAPACITY, INCREASE_CAPACITY y REDUCE_CAPACITY.
- Ejecutar acciones reales de creación, registro, desregistro y terminación de instancias EC2.
- Respetar una capacidad mínima de 1 y máxima de 5 instancias en todo momento.
- Incorporar un cooldown de 300 segundos y un periodo de gracia de warm-up de 120 segundos para evitar oscilaciones y decisiones basadas en métricas no confiables.
- Ejecutar el controlador de forma continua como servicio systemd, con reinicio automático ante fallos.
- Validar experimentalmente los tres escenarios de decisión, la ventana anti-oscilación y el comportamiento del sistema ante un fallo real de ejecución.

3. Arquitectura implementada

La arquitectura se dividió en dos planos independientes. El plano de datos está compuesto por el ALB, el Target Group y las instancias EC2 que sirven la aplicación web. El plano de control es una instancia EC2 separada, con un tag distinto al de las instancias web, que ejecuta el controlador como servicio systemd y opera sobre AWS mediante el rol IAM LabInstanceProfile/LabRole (sin credenciales estáticas).



3.1 Componentes principales

Componente	Configuración	Función
Application Load Balancer	webserver-alb webserver-alb-1918446635.us-east-1.elb.amazonaws.com	Punto de entrada HTTP y distribución de tráfico entre AZs.
Target Group	webserver-tg, HTTP:80, health check /health/	Mantiene los backends web y sus health checks (umbral 2, intervalo 15s).
Instancias web	t2.micro, Amazon Linux 2023, tag Role=webserver	Capacidad que el controlador crea o elimina dinámicamente.
Launch Template	webserver-template (lt-06e3c86a43e610770)	Plantilla usada para lanzar instancias web homogéneas.
CloudWatch	CPUUtilization (decisión) y TargetResponseTime (contexto)	Fuente de métricas del ciclo Observar.
EC2 controladora	Instancia independiente, systemd	Ejecuta controller.py en loop continuo cada 60s.
IAM	LabInstanceProfile / LabRole	Permite llamadas a AWS sin credenciales estáticas en el código.
Ejecución periódica	systemd (Restart=on-failure)	Mantiene el controlador vivo; reinicia el proceso ante caídas.

A diferencia de diseños que protegen una instancia base mediante una etiqueta explícita (por ejemplo Protected=true), este controlador no distingue ninguna instancia como intocable más allá del límite global MIN_INSTANCES=1. La Sección 8 muestra por qué, combinado con otro defecto, esto tuvo consecuencias reales durante la validación.

4. Preparación de la infraestructura

4.1 Red y seguridad

Se utilizó la VPC por defecto (vpc-09e37bed13ef17dcb) con dos subredes en distintas zonas de disponibilidad: subnet-0aadc82eace6d0dde (us-east-1a) y subnet-036da28cea6fbe467 (us-east-1b). Se crearon tres Security Groups:

- SG-ALB (sg-037d546de26aa6147): permite HTTP 80 desde 0.0.0.0/0.
- SG-Webserver (sg-03430922ea8e4fd76): permite HTTP 80 únicamente desde SG-ALB, y SSH 22 desde la IP de administración.
- SG-Controller (sg-0efabefa2b389e194): permite SSH 22 desde la IP de administración; el tráfico saliente está abierto para poder llamar a las APIs de AWS.

Security Groups (4) Info

Find security groups by attribute or tag

Actions

Export security groups to CSV

Create security group

Security group ID	Security group name	VPC ID	Description	Owner	Inbound rules count
sg-037d546de26aa6147	SG-ALB	vpc-09e37bed13ef17dcb	Permite HTTP publico hacia el Load Bal...	940114063581	1 Permission entry
sg-0548ad48f6d6bb2ae	default	vpc-09e37bed13ef17dcb	default VPC security group	940114063581	1 Permission entry
sg-03430922ea8e4fd76	SG-Webserver	vpc-09e37bed13ef17dcb	Instancias nginx, solo reciben trafico del...	940114063581	2 Permission entries
sg-0efabefa2b389e194	SG-Controller	vpc-09e37bed13ef17dcb	Instancia que corre el auto-scaling contr...	940114063581	1 Permission entry

4.2 Servidor web

Las instancias web se aprovisionan mediante un script de user-data que instala nginx sobre Amazon Linux 2023 y publica tres rutas: '/' (página de verificación con instance-id y AZ), '/health/' (endpoint liviano para el health check del ALB) y '/stress/?n=<iteraciones>' (endpoint que ejecuta hashing SHA-256 repetido, corriendo como servicio systemd independiente en 127.0.0.1:8080 y expuesto vía proxy_pass de nginx). Este último endpoint fue necesario porque servir contenido estático es computacionalmente casi gratuito para nginx, y generar suficiente concurrencia para saturar la CPU real de una t2.micro desde un generador de carga de laboratorio (ab desde WSL2) resulta poco práctico contra contenido tan liviano.

```
egoq@MSI:~$ curl "http://webserver-alb-1918446635.us-east-1.elb.amazonaws.com/health/"
OK
```

4.3 Target Group y Load Balancer

El Target Group webserver-tg se configuró en HTTP:80 con health check sobre la ruta /health/, umbral healthy/unhealthy de 2 e intervalo de 15 segundos. El ALB webserver-alb se configuró como internet-facing, usando las dos subredes en AZs distintas y el Security Group SG-ALB, con un listener HTTP:80 que reenvía al Target Group.

webserver-tg

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:940114063581:targetgroup/webserver-tg/83efde1875ab9b80

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Load balancer

webserver-alb

Protocol version

HTTP1

VPC

vpc-09e37bed13ef17dcb

2

Total targets

1

Healthy

0 Anomalous

0

Unhealthy

0

Unused

0

Initial

1

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Anomaly mitigation: Not applicable

Deregister

Register target

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

Instance ID

Name

Port

Zone

Health status

Health status details

Admin...

Overri...

Launch...

Anomaly detection

i-03ba0f0ec9116f6

-

80

Draining

Target deregistration is in progress

No override

No overvi...

Normal

i-08679e9f32121a7e3

-

80

us-east-1a (us-...

Healthy

No override

No overvi...

September...

Normal

4.4 Launch Template

El Launch Template `webserver-template (lt-06e3c86a43e610770)` define AMI Amazon Linux 2023, tipo `t2.micro`, el Security Group `SG-Webserver`, el perfil de instancia `LabInstanceProfile` y el `user-data` descrito en 4.2. La subred no se fija en la plantilla: el controlador la especifica en tiempo de lanzamiento mediante la variable `SUBNET_ID`. Esto implica una limitación real: todas las instancias que lanza el controlador caen siempre en la misma AZ configurada en `.env`, lo cual se documenta en la Sección 12.

webserver-template (lt-06e3c86a43e610770)

ActionsDelete template

Launch template details

Launch template ID
lt-06e3c86a43e610770

Launch template name
webserver-template

Default version
1

Owner
arn:aws:sts::940114063581:assumed-role/voclabs/user4073363=EMANUEL_GONZALEZ_QUINTERO

DetailsVersionsTemplate tags

Launch template version details

Version
1 (Default)

Description
nginx + endpoint de health check + stress endpoint

Date created
2026-09-22T07:39:20.000Z

Created by
arn:aws:sts::940114063581:assumed-role/voclabs/user4073363=EMANUEL_GONZALEZ_QUINTERO

Instance detailsStorageResource tagsNetwork interfacesAdvanced details

Instances (6) Info

Last updated 14 minutes agoConnectInstance stateActionsLaunch instances

Saved filter setsChoose filter setFilter Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐		i-0fed8e2eb3494c30d	Terminated	t2.micro	–	us-east-1a	–
☐		i-0564f71997834bc72	Terminated	t2.micro	–	us-east-1a	–
☐		i-08676e9f32121a7e3	Running	t2.micro	2/2 checks passed	us-east-1a	ec2-100-48-87-7.comp...
☐		i-07cdd47465efa2fb7	Terminated	t2.micro	–	us-east-1a	–
☐	autoscaling-controller	i-0ec5709e3f5fc5cb1	Running	t2.micro	2/2 checks passed	us-east-1a	ec2-3-80-26-241.comp...
☐		i-03be0f0eec91f6ffc	Terminated	t2.micro	–	us-east-1a	–

5. Diseño del controlador personalizado

El controlador está implementado en Python con `boto3` y sigue el ciclo Observar → Filtrar warm-up → Analizar → Decidir → Actuar, ejecutado cada 60 segundos (`EVALUATION_INTERVAL_SECONDS`).

5.1 Observar

`describe_managed_instances()` descubre las instancias mediante el tag `Role=webserver` en estado `running` o `pending`. `get_target_health()` consulta el estado de cada target en el ALB. `get_cpu_utilization_per_instance()` consulta CloudWatch (`GetMetricData`, estadístico `Average`) con una ventana de `lookback` de 6 minutos y periodo de 60 segundos. `TargetResponseTime` del ALB se consulta y se registra en cada decisión como métrica de contexto, pero no participa en la lógica de decisión: en este diseño la señal de escalamiento es exclusivamente la CPU de la flota.

5.2 Filtrado de warm-up

Una instancia recién lanzada se registra en `instancias_en_warmup` con su timestamp de lanzamiento. Se considera 'caliente' (confiable para decisiones) solo cuando el ALB la reporta `healthy` Y ha transcurrido el periodo de gracia `WARMUP_GRACE_SECONDS=120s` desde su lanzamiento. La Sección 8 documenta un defecto real encontrado en esta función durante la validación, y su corrección.

5.3 Analizar y decidir

Condición	Decisión	Justificación
CPU promedio de la flota > 70% durante 2 evaluaciones consecutivas (~2 min) y capacidad_actual < 5	INCREASE_CAPACITY	Ventana corta: proteger la disponibilidad reaccionando rápido ante sobrecarga.
CPU promedio de la flota < 30% sostenida durante 5 evaluaciones consecutivas (~5 min) y capacidad_actual > 1	REDUCE_CAPACITY	Ventana larga: evitar remover capacidad ante caídas transitorias de tráfico.
CPU dentro de [30%, 70%], condición no sostenida, o en cooldown	MAINTAIN_CAPACITY	Zona muerta de histéresis; cooldown de 300s tras cualquier acción evita flapping.

5.4 Actuar

En INCREASE_CAPACITY: se lanza una instancia desde el Launch Template, se registra en el Target Group y se agrega a instancias_en_warmup. En REDUCE_CAPACITY: se selecciona la instancia con menor CPU entre las instancias válidas, se desregistra del Target Group, se espera TARGET_DEREGISTRATION_DRAIN_SECONDS=60s para permitir el drenaje de conexiones, y finalmente se termina. No existe una etiqueta de protección explícita sobre ninguna instancia particular; el único resguardo es el límite global MIN_INSTANCES.

5.5 Resiliencia

Cada fase (Observar, Actuar) está envuelta en manejo de excepciones independiente. Los errores transitorios de AWS (Throttling, RequestLimitExceeded, InternalError) se reintentan con backoff fijo; los errores de permisos o parámetros no se reintentan. Se llevan contadores de fallos consecutivos de observación y de acción, con un umbral crítico de 3 que emite una alerta de nivel CRITICAL en los logs. Si una acción falla, deliberadamente no se activa el cooldown, para permitir un reintento inmediato en el siguiente ciclo.

6. Automatización del controlador

El controlador corre como servicio systemd (autoscaling-controller.service) en una instancia EC2 dedicada, en lugar de una tarea cron periódica. controller.py implementa su propio loop continuo (while True) con una espera de EVALUATION_INTERVAL_SECONDS=60 ajustada por la duración real del ciclo anterior, y systemd garantiza el reinicio automático del proceso ante una caída (Restart=on-failure, RestartSec=10).

```
[Unit]
Description=Auto-Scaling Controller (SI3016 Challenge Based Learning No. 1)
After=network-online.target

[Service]
Type=simple
User=ec2-user
WorkingDirectory=/home/ec2-user/controller-challenge
EnvironmentFile=/home/ec2-user/controller-challenge/.env
ExecStart=/usr/bin/python3 /home/ec2-user/controller-challenge/controller.py
Restart=on-failure
RestartSec=10
```

Toda la salida técnica se registra en logs/controller.log, y cada ciclo de decisión se registra como una línea JSON estructurada en logs/decisions.log, lo que permitió el análisis detallado presentado en las Secciones 7 y 8.

7. Validación experimental

Todas las pruebas se ejecutaron sobre infraestructura real, con acciones habilitadas (sin simulaciones). El generador de carga fue Apache Bench (ab) desde una máquina local, apuntando al endpoint /stress/ descrito en la Sección 4.2.

7.1 Ciclo base sin carga

Con una sola instancia web activa (i-07cdd47465efa2fb7) y healthy, el controlador registró ciclos consecutivos de MAINTAIN_CAPACITY con CPU estable entre 1.4% y 1.5%, confirmando el funcionamiento correcto del ciclo Observar-Analizar-Decidir en ausencia de carga.

```
{
  "timestamp": "2026-09-22T10:55:03.088683+00:00",
  "instancias_totales": [
    {
      "i-08676e9f32121a7e3": "healthy",
      "i-06d356acd021c527e": "healthy",
      "salud_alb": {
        "i-06d356acd021c527e": "healthy",
        "i-08676e9f32121a7e3": "healthy"
      },
      "cpu_por_instancia": {
        "i-08676e9f32121a7e3": 2.3333333333333335,
        "i-06d356acd021c527e": 1.332800213248034
      },
      "latencia_alb_promedio_ms": null,
      "instancias_validas_para_decision": [
        "i-08676e9f32121a7e3",
        "i-06d356acd021c527e"
      ],
      "instancias_en_warmup": [],
      "capacidad_actual": 2,
      "cpu_promedio_flota": 1.83,
      "ventana_evaluada_subida": [
        {
          "timestamp": "2026-09-22T10:54:03.251521+00:00",
          "valor": 1.858580098706149
        },
        {
          "timestamp": "2026-09-22T10:55:03.285524+00:00",
          "valor": 1.8330667732906838
        }
      ],
      "ventana_evaluada_bajada": [
        {
          "timestamp": "2026-09-22T10:51:03.088467+00:00",
          "valor": 54.111666666666665
        },
        {
          "timestamp": "2026-09-22T10:52:03.131562+00:00",
          "valor": 1.975
        },
        {
          "timestamp": "2026-09-22T10:53:03.276928+00:00",
          "valor": 1.8830534452885512
        },
        {
          "timestamp": "2026-09-22T10:54:03.251521+00:00",
          "valor": 1.858580098706149
        },
        {
          "timestamp": "2026-09-22T10:55:03.285524+00:00",
          "valor": 1.8330667732906838
        }
      ],
      "decision": "MAINTAIN_CAPACITY",
      "justificacion": "CPU promedio actual (1.83%) dentro de la zona muerta [30.0, 70.0] o la condicion aun no es sostenida en la ventana requerida.",
      "accion_solicitada": "NONE",
      "resultado_accion": "N/A"
    }
  ],
  "timestamp": "2026-09-22T10:56:03.118698+00:00",
  "instancias_totales": [
    {
      "i-08676e9f32121a7e3": "healthy",
      "i-06d356acd021c527e": "healthy",
      "salud_alb": {
        "i-06d356acd021c527e": "healthy",
        "i-08676e9f32121a7e3": "healthy"
      },
      "cpu_por_instancia": {
        "i-08676e9f32121a7e3": 2.416666666666667,
        "i-06d356acd021c527e": 1.267173536081099
      },
      "latencia_alb_promedio_ms": null,
      "instancias_validas_para_decision": [
        "i-08676e9f32121a7e3",
        "i-06d356acd021c527e"
      ],
      "instancias_en_warmup": [],
      "capacidad_actual": 2,
      "cpu_promedio_flota": 1.84,
      "ventana_evaluada_subida": [
        {
          "timestamp": "2026-09-22T10:55:03.285524+00:00",
          "valor": 1.8330667732906838
        },
        {
          "timestamp": "2026-09-22T10:56:03.416515+00:00",
          "valor": 1.841920101373883
        }
      ],
      "ventana_evaluada_bajada": [
        {
          "timestamp": "2026-09-22T10:52:03.131562+00:00",
          "valor": 1.975
        },
        {
          "timestamp": "2026-09-22T10:53:03.276928+00:00",
          "valor": 1.8830534452885512
        },
        {
          "timestamp": "2026-09-22T10:54:03.251521+00:00",
          "valor": 1.858580098706149
        },
        {
          "timestamp": "2026-09-22T10:55:03.285524+00:00",
          "valor": 1.8330667732906838
        },
        {
          "timestamp": "2026-09-22T10:56:03.416515+00:00",
          "valor": 1.841920101373883
        }
      ],
      "decision": "REDUCE_CAPACITY",
      "justificacion": "CPU promedio de la flota < 30.0% sostenida en 5 evaluaciones consecutivas: [1.975, 1.8830534452885512, 1.858580098706149, 1.8330667732906838, 1.841920101373883]",
      "accion_solicitada": "deregister_targets + terminate_instances (i-06d356acd021c527e)",
      "resultado_accion": "OK - instancia terminada: i-06d356acd021c527e"
    }
  ],
  "timestamp": "2026-09-22T10:57:04.437456+00:00",
  "instancias_totales": [
    {
      "i-08676e9f32121a7e3": "healthy",
      "i-06d356acd021c527e": "healthy",
      "salud_alb": {
        "i-06d356acd021c527e": "draining",
        "i-08676e9f32121a7e3": "healthy"
      },
      "cpu_por_instancia": {
        "i-08676e9f32121a7e3": 2.1833333333333333,
        "i-06d356acd021c527e": 1.3333333333333335
      },
      "latencia_alb_promedio_ms": null,
      "instancias_validas_para_decision": [
        "i-08676e9f32121a7e3",
        "i-06d356acd021c527e"
      ],
      "instancias_en_warmup": [
        "i-06d356acd021c527e"
      ],
      "capacidad_actual": 2,
      "cpu_promedio_flota": 2.18,
      "ventana_evaluada_subida": [
        {
          "timestamp": "2026-09-22T10:56:03.416515+00:00",
          "valor": 1.841920101373883
        },
        {
          "timestamp": "2026-09-22T10:57:04.598453+00:00",
          "valor": 2.1833333333333333
        }
      ],
      "ventana_evaluada_bajada": [
        {
          "timestamp": "2026-09-22T10:53:03.276928+00:00",
          "valor": 1.8830534452885512
        },
        {
          "timestamp": "2026-09-22T10:54:03.251521+00:00",
          "valor": 1.858580098706149
        },
        {
          "timestamp": "2026-09-22T10:55:03.285524+00:00",
          "valor": 1.8330667732906838
        },
        {
          "timestamp": "2026-09-22T10:56:03.416515+00:00",
          "valor": 1.841920101373883
        },
        {
          "timestamp": "2026-09-22T10:57:04.598453+00:00",
          "valor": 2.1833333333333333
        }
      ],
      "decision": "MAINTAIN_CAPACITY",
      "justificacion": "En cooldown desde la ultima accion (REDUCE_CAPACITY); faltan 299s para poder volver a actuar.",
      "accion_solicitada": "NONE",
      "resultado_accion": "N/A"
    }
  ]
}
```

7.2 Calibración de la carga sintética

Una prueba manual con curl ".../stress/?n=100000" respondió en 0.311 segundos totales (incluyendo la ida y vuelta por Internet a través del ALB), lo que indicó que ese valor de n consumía muy poca CPU real. Tras calibrar con valores mayores, se utilizó n=1500000 para lograr una saturación significativa de una t2.micro (instancia burstable, con créditos de CPU limitados).

7.3 Escenario de sobrecarga sostenida

Con ab -k -n 5000 -c 50 apuntando a /stress/?n=1500000, la CPU promedio de la flota pasó de 1.7% a 35.7% y luego a 99.59% en dos evaluaciones consecutivas, cumpliendo la condición de INCREASE_CAPACITY (CPU > 70% en 2 evaluaciones). El controlador tomó correctamente la decisión de aumentar capacidad; sin embargo, la ejecución de esa acción reveló un defecto real de ejecución, documentado en detalle en la Sección 8.

```

ago09M$- $ ab -k -n 5000 -c 50 "http://webserv
ers-alb-191846635.us-east-1.elb.amazonaws.com/stress/?n=900800"
This is ApacheBench, Version 2.3 <Revision: 19
03618 >
Copyright 1996 Adam Twiss, Zeus Technology Ltd,
http://www.zestech.net.
Licensed to The Apache Software Foundation, htt
p://www.apache.org/

Benchmarking webservers-alb-191846635.us-east-
1.elb.amazonaws.com (be patient)
Completed 500 requests
Completed 1000 requests
^C

Server Software:      nginx/1.30.4
Server Hostname:      webservers-alb-19184664
35.us-east-1.elb.amazonaws.com
Server Port:          80

Document Path:         /stress/?n=900800
Document Length:       3 bytes

Concurrency Level:     50
Time taken for tests:   553.145 seconds
Complete requests:      1017
Failed requests:        15
    (Connect: 0, Receive: 0, Length: 15, Excepti
ons: 0)
Non-2xx responses:     15
Keep-Alive requests:    15
Total transferred:      138632 bytes
HTML transferred:       4986 bytes
Requests per second:    1.814 [#sec] (mean)
Time per request:       27194.933 [ms] (mean)
Time per request:       544.899 [ms] (mean, acr
oss all concurrent requests)
Transfer rate:           0.23 [Kbytes/sec] recei

```

8. Incidente real de resiliencia: condición de carrera y falla en cascada

Esta sección documenta, con datos reales de `decisions.log`, una falla genuina detectada durante la validación experimental, su efecto en cascada sobre la disponibilidad del servicio, y la corrección aplicada. Se incluye deliberadamente como evidencia de análisis de causa raíz, más allá de una validación de 'camino feliz'.

8.1 Disparador: condición de carrera en el registro de instancias

Al ejecutar `INCREASE_CAPACITY`, `aws_client.launch_instance()` lanzó correctamente una nueva instancia (`i-0564f71997834bc72`), pero `register_target()` se invocó inmediatamente después, mientras la instancia aún estaba en estado `pending` en EC2. La API de Target Groups rechaza el registro de instancias que no están en estado `running`, por lo que la acción falló:

```
resultado_accion: "FALL0: An error occurred (InvalidTarget) when calling
the RegisterTargets operation: The following targets are not in a
running state and cannot be registered: 'i-0564f71997834bc72'"
```

8.2 Efecto en cascada: instancia huérfana en la toma de decisiones

Como la excepción ocurrió antes de que se ejecutara `sm.add_instance_to_warmup()`, la nueva instancia nunca quedó registrada como 'en warm-up'. En el siguiente ciclo, describe_managed_instances() ya la reportaba como running, y `is_instance_warm()` —al no encontrarla en el tracking de warm-up— la consideraba válida incondicionalmente, sin verificar si realmente estaba healthy en el ALB. El resultado fue que una instancia nunca registrada en el Target Group, con 0% de CPU real, entró al cálculo del promedio de la flota, diluyéndolo de 99.59% a 49.78% en el ciclo siguiente y ocultando que la instancia real seguía saturada.

8.3 Consecuencia: terminación de la instancia equivocada

Minutos después, con la CPU de ambas instancias ya baja, se cumplió la condición de REDUCE_CAPACITY (5 evaluaciones consecutivas por debajo de 30%). El algoritmo de selección eligió la instancia con menor CPU entre las 'válidas': por una diferencia mínima (1.45% contra 1.97%), esa instancia resultó ser la real (i-07cdd47465efa2fb7), no la huérfana. El controlador la desregistró y la terminó exitosamente, dejando como única instancia viva a la huérfana, que nunca había sido registrada en el Target Group.

8.4 Impacto: caída total del servicio

Desde ese momento, `salud_alb` quedó vacío en todos los ciclos posteriores: el Target Group no tenía ningún target saludable, y el ALB no tenía a quién enviar tráfico. El sistema quedó en un estado estable pero inútil: la única instancia viva nunca recibía tráfico (CPU baja permanente), por lo que jamás se cumpliría la condición de `INCREASE_CAPACITY` para reemplazar la capacidad perdida. Sin intervención, la caída habría sido indefinida.

8.5 Recuperación

Se registró manualmente la instancia huérfana (`i-0564f71997834bc72`) en el Target Group desde la consola de AWS. Su servicio `nginx` ya estaba activo (el `user-data` se ejecuta al arrancar, independientemente del registro en el ALB), por lo que pasó a `healthy` tras los health checks y el servicio quedó restaurado.

8.6 Corrección de causa raíz

Se aplicaron dos cambios de código, ambos con impacto directo en el mecanismo que causó el incidente:

a) `aws_client.py` — esperar el estado `running` antes de registrar el target:

```
def launch_instance():
    response = _with_retries(ec2.run_instances, ...)
    instance_id = response['Instances'][0]['InstanceId']
    waiter = ec2.get_waiter('instance_running')
    waiter.wait(InstanceIds=[instance_id],
                WaiterConfig={'Delay': 5, 'MaxAttempts': 24})
    return instance_id
```

b) `state_manager.py` — no asumir válida una instancia no trackeada:

```
def is_instance_warm(state, instance_id, is_healthy, warmup_grace_seconds):
    launched_at = state['instancias_en_warmup'].get(instance_id)
    if launched_at is None:
        return is_healthy # antes: return True (bug)
    ...
```

El primer cambio elimina el disparador (la condición de carrera). El segundo elimina el mecanismo de propagación: incluso si una instancia queda huérfana por cualquier otra causa futura, nunca será tratada como válida para el cálculo de CPU ni como candidata de reducción de capacidad mientras no esté realmente `healthy` en el ALB.

8.7 Discusión

Este incidente evidencia un principio general de sistemas de control en la nube: la validez de un recurso para efectos de decisión debe basarse siempre en su estado verificado en tiempo real, nunca en la ausencia de información en contrario. También sugiere una mejora complementaria: un mecanismo de reconciliación periódica que compare instancias con el tag esperado contra los targets efectivamente registrados en el ALB, para detectar y limpiar huérfanos de forma proactiva en vez de depender únicamente de que el algoritmo de decisión las descarte (ver Sección 12).

9. Repetición de las pruebas tras la corrección

Tras aplicar ambos cambios y reiniciar el servicio con el estado local reinicializado, se repitieron los tres escenarios de validación para confirmar que el defecto quedó resuelto.

9.1 Escenario de sobrecarga sostenida (repetición)

[illegible]

9.2 Escenario de subutilización sostenida (repetición)

[illegible]

9.3 Escenario de pico transitorio (anti-oscilación)

```
Benchmarking webserver-alb-1918446635.us-east-1.elb.amazonaws.com (be patient)
```

```
00000000, value: 2.4833333333333333, { timestamp: 2026-09-22T10:44:02.8867355+00:00, value: 2.7833333333333333, {"timestamp": "2026-09-22T10:45:02.853590+00:00", "value": 2.4499999999999997, {"timestamp": "2026-09-22T10:46:02.937976+00:00", "value": 2.2666666666666666}, {"decision": "MAINTAIN CAPACITY", "just
```

- Los umbrales de CPU (30%/70%) son estáticos y se calibraron para un entorno de laboratorio; en producción deberían derivarse de tráfico histórico, objetivos de latencia y costo.
- La decisión se basa únicamente en CPU. TargetResponseTime se registra pero no participa en la lógica; incorporar señales adicionales (tasa de errores HTTP, latencia, RequestCount) permitiría distinguir mejor entre carga computacional y crecimiento de tráfico.
- El estado (cooldown, warm-up, historial de CPU) se guarda localmente en state.json en la instancia controladora; una versión de alta disponibilidad debería usar almacenamiento externo para tolerar el reemplazo de esa instancia.
- SUBNET_ID es fijo en la configuración, por lo que todas las instancias lanzadas por el controlador caen siempre en la misma AZ, reduciendo la resiliencia multi-AZ real del auto-escalado.
- No existe un mecanismo de reconciliación periódica que detecte instancias con el tag esperado que nunca llegaron a registrarse en el Target Group. El incidente de la Sección 8 se corrigió en su síntoma

(is_instance_warm ya no las asume válidas), pero una reconciliación activa que las limpie de forma proactiva sería más robusta.

- A diferencia de diseños que protegen explícitamente una instancia base con una etiqueta (por ejemplo Protected=true), este controlador no distingue ninguna instancia como no-eliminable más allá del límite global MIN_INSTANCES; incorporar ese patrón habría añadido una capa adicional de seguridad frente al incidente descrito.
- wait_for_drain() en aws_client.py contiene una condición redundante que conviene simplificar en una futura revisión de código, aunque no afectó el resultado de las pruebas realizadas.

13. Conclusiones

Se implementó un controlador de auto-escalado horizontal propio, capaz de observar CPUUtilization vía CloudWatch, decidir explícitamente entre tres estados mediante histéresis con ventanas asimétricas, y ejecutar acciones reales sobre EC2 y un Target Group de ALB. Los experimentos confirmaron los tres comportamientos esperados: mantenimiento en zona muerta, incremento ante sobrecarga sostenida y reducción ante subutilización sostenida, respetando en todo momento los límites de capacidad [1,5] y el cooldown de 300 segundos.

Más allá del camino esperado, la validación expuso una condición de carrera real entre el ciclo de vida de EC2 y el registro en el Target Group, cuya propagación —facilitada por un supuesto incorrecto en la lógica de warm-up— provocó una caída total y real del servicio. El diagnóstico de causa raíz, la corrección aplicada a dos módulos del controlador y la repetición exitosa de las pruebas constituyen, en conjunto, una evidencia más completa de la robustez (y de los límites) del diseño que una validación libre de incidentes.

14. Anexo A. Datos técnicos relevantes

Elemento	Valor
Región	us-east-1
VPC	vpc-09e37bed13ef17dcb
Subred AZ us-east-1a	subnet-0aadc82eace6d0dde
Subred AZ us-east-1b	subnet-036da28cea6fbe467
SG-ALB	sg-037d546de26aa6147
SG-Webserver	sg-03430922ea8e4fd76
SG-Controller	sg-0efabefa2b389e194
Target Group (ARN)	arn:aws:elasticloadbalancing:us-east-1:940114063581:targetgroup/webserver-tg/83efde1875ab9b80
ALB (DNS)	webserver-alb-1918446635.us-east-1.elb.amazonaws.com
Launch Template	lt-06e3c86a43e610770
Instancia web original	i-07cdd47465efa2fb7 (terminada durante el incidente de la Sección 8)
Instancia involucrada en el incidente	i-0564f71997834bc72 (huérfana, luego recuperada)
Tipo de instancias	t2.micro
Capacidad	mínimo 1, máximo 5
Umbral scale-up / scale-down	CPU > 70% / CPU < 30%
Evaluaciones consecutivas	2 (subida) / 5 (bajada)
Cooldown	300 segundos
Warm-up grace	120 segundos
Frecuencia de evaluación	60 segundos (systemd, loop continuo)